

Database Integration

Connect, Query, and Manage Data Professionally

■ LEARNING OUTCOME

By the end of this module you will integrate PostgreSQL and MongoDB into Node.js backends using Prisma ORM, write efficient queries, design schemas, handle migrations, and implement proper data access patterns.

■ Skills You Will Gain

- Design a relational database schema with proper normalisation
- Set up Prisma ORM with PostgreSQL and run migrations
- Write CRUD operations using Prisma Client
- Design MongoDB schemas with Mongoose
- Implement indexes for query performance
- Use environment variables for secure database connections

■ Database integration is where most junior projects break down. Proper schema design, migrations, and indexed queries are what separate a working side project from a production-ready application.

SECTION 1

Schema Design Principles

Database Design Rules

Normalisation -- eliminate data redundancy: 1NF: Each column has atomic (indivisible) values. No repeating groups. 2NF: 1NF + no partial dependencies (every column depends on full primary key). 3NF: 2NF + no transitive dependencies (A->B->C; B should be its own table). Common patterns: One-to-Many: User has many Tasks (userId FK on tasks table) Many-to-Many: Task has many Tags, Tag on many Tasks (join table: task_tags) One-to-One: User has one Profile (userId FK on profiles table, unique) Always include: created_at, updated_at timestamps on every table. Always use: UUID or auto-increment integer as primary key.

```
# PRISMA SETUP npm install prisma @prisma/client npx prisma init # creates prisma/schema.prisma and .env # .env DATABASE_URL='postgresql://user:password@localhost:5432/taskdb' # prisma/schema.prisma generator client { provider = 'prisma-client-js' } datasource db { provider = 'postgresql' url = env('DATABASE_URL') } model User { id String @id @default(uuid()) email String @unique name String password String role Role @default(USER) tasks Task[] // one-to-many createdAt DateTime @default(now()) updatedAt DateTime @updatedAt @@index([email]) } model Task { id String @id @default(uuid()) title String @db.VarChar(200) description String? @db.Text status Status @default(TODO) priority Priority @default(MEDIUM) dueDate DateTime? user User @relation(fields: [userId], references: [id], onDelete: Cascade) userId String tags TaskTag[] createdAt DateTime @default(now()) updatedAt DateTime @updatedAt @@index([userId]) // index for foreign key queries @@index([userId, status]) // compound index for filtered queries } model Tag { id String @id @default(uuid()) name String @unique tasks TaskTag[] } model TaskTag { // many-to-many join table task Task @relation(fields: [taskId], references: [id], onDelete: Cascade) taskId String tag Tag @relation(fields: [tagId], references: [id], onDelete: Cascade) tagId String @@id([taskId, tagId]) // composite primary key } enum Role { USER ADMIN } enum Status { TODO IN_PROGRESS DONE } enum Priority { LOW MEDIUM HIGH } # MIGRATIONS npx prisma migrate dev --name init # create and apply migration npx prisma migrate deploy # apply migrations in production npx prisma db push # sync schema without migration (dev only) npx prisma studio # visual database browser
```

SECTION 2

Prisma Client Queries

```
const { PrismaClient } = require('@prisma/client'); const prisma = new PrismaClient({ log: process.env.NODE_ENV === 'development' ? ['query'] : [], }); // CREATE const task = await prisma.task.create({ data: { title: 'Build portfolio site', priority: 'HIGH', userId: user.id, tags: { create: [ // create and connect tags in one query { tag: { connectOrCreate: { where:{name:'React'}, create:{name:'React'} } } }, ], }, }, include: { tags: { include: { tag: true } } }, }); // READ -- with filtering, sorting, pagination, and includes const result = await prisma.task.findMany({ where: { userId: userId, status: { not: 'DONE' }, // not equal priority: { in: ['HIGH','MEDIUM'] }, dueDate: { lte: new Date() }, // overdue tasks title: { contains: searchTerm, mode: 'insensitive' }, }, orderBy: [{ priority: 'desc' }, { dueDate: 'asc' }], skip: (page - 1) * limit, take: limit, include: { tags: { include: { tag: true } }, user: { select: { id:true, name:true, email:true } }, // select specific fields }, }); // UPDATE -- atomic operations const updated = await prisma.task.update({ where: { id: taskId, userId: userId }, // ownership check in query! data: { status: 'DONE', updatedAt: new Date(), }, }); // UPSERT -- create or update const tag = await prisma.tag.upsert({ where: { name: tagName }, update: {}, create: { name: tagName }, }); // TRANSACTION -- atomic multi-step operations const [newTask, updatedUser] = await prisma.$transaction([ prisma.task.create({ data: taskData }), prisma.user.update({ where:{id:userId}, data:{taskCount:{increment:1}} }, ), ]); // AGGREGATE const stats = await prisma.task.groupBy({ by: ['status'], _count: { id: true }, where: { userId }, }); // Returns: [{status:'TODO',_count:{id:5}},{status:'DONE',_count:{id:12}}]
```

MongoDB with Mongoose

```
npm install mongoose // config/database.js
const mongoose = require('mongoose');
const connectDB = async () => {
  await mongoose.connect(process.env.MONGODB_URI, {
    maxPoolSize: 10, // connection pooling
  });
  console.log('MongoDB connected');
};
module.exports = connectDB;

// models/Task.js
const { Schema, model } = require('mongoose');
const taskSchema = new Schema({
  title: { type: String, required: [true, 'Title required'], maxLength: 200, trim: true },
  description: { type: String },
  status: { type: String, enum: ['todo', 'in-progress', 'done'], default: 'todo' },
  priority: { type: String, enum: ['low', 'medium', 'high'], default: 'medium' },
  dueDate: { type: Date },
  tags: [{ type: String }],
  user: { type: Schema.Types.ObjectId, ref: 'User', required: true },
}, { timestamps: true });

// INDEXES
taskSchema.index({ user: 1, status: 1 }); // compound index
taskSchema.index({ dueDate: 1 }); // for overdue queries
taskSchema.index({ title: 'text' }); // full-text search

// MIDDLEWARE
taskSchema.pre('save', function(next) {
  if (this.isModified('status') && this.status === 'done') {
    this.completedAt = new Date();
  }
  next();
});

// VIRTUAL -- computed field not stored in DB
taskSchema.virtual('isOverdue').get(function() {
  return this.dueDate && this.dueDate < new Date() && this.status !== 'done';
});

module.exports = model('Task', taskSchema);

// Mongoose queries
const tasks = await Task.find({ user: userId, status: 'todo' })
  .sort({ priority: -1, dueDate: 1 })
  .skip((page-1)*limit).limit(limit)
  .populate('user', 'name email')
  .lean(); // returns plain JS objects, faster than Mongoose docs
```

■ PRACTICE Q & A

Q1. What is the difference between SQL and NoSQL databases?

A: *SQL (PostgreSQL, MySQL): structured tables with predefined schema, ACID transactions, complex joins, great for relational data. NoSQL (MongoDB): flexible documents, horizontal scale, great for hierarchical/varied data. Choose SQL for most apps -- NoSQL when you have truly variable schemas or massive scale.*

Q2. What is a database migration and why is it important?

A: *A versioned, reproducible script that changes the database schema. Like Git for your database. Run the same migrations on dev, staging, and production to keep schemas in sync. Never modify production databases directly -- always use migrations.*

Q3. What is an index and when should you add one?

A: *An index creates a sorted data structure on a column for fast lookups. Add indexes on: foreign keys, columns in WHERE clauses, columns in ORDER BY. Cost: extra storage and slower writes. Rule: index columns you filter or sort by frequently.*

Q4. What is Prisma and what problem does it solve?

A: *Type-safe ORM that generates a client from your schema. Advantages: TypeScript autocomplete for queries, automatic migrations, prevents SQL injection, and readable query syntax. Alternatives: Sequelize (Node), SQLAlchemy (Python), Hibernate (Java).*

Q5. What is connection pooling?

A: *Maintaining a pool of reusable database connections rather than creating a new connection per request. Creating connections is expensive (100-300ms). A pool of 10 connections can handle hundreds of concurrent requests efficiently.*

■ Database Cheat Sheet

<code>npm prisma init</code>	Create Prisma setup files
<code>npm prisma migrate dev --name x</code>	Create + apply migration
<code>npm prisma studio</code>	Visual database browser
<code>prisma.task.findMany({where, orderBy, skip, take})</code>	Query with filter+sort+pagination
<code>include: { user: true }</code>	Join related records
<code>select: { id: true, name: true }</code>	Select specific fields only
<code>prisma.\$transaction([p1, p2])</code>	Atomic multi-query transaction
<code>connectOrCreate</code>	Create or connect relation

<code>@@index([userId, status])</code>	Compound index in Prisma schema
<code>mongoose.model('Task', schema)</code>	Create Mongoose model
<code>.lean()</code>	Return plain JS objects (faster)
<code>{ timestamps: true }</code>	Auto created_at + updated_at